

Robust Interplanetary Trajectory Design under Multiple Uncertainties via Meta-Reinforcement Learning

Lorenzo Federici^{a,*}, Alessandro Zavoli^b

^a*Department of Systems & Industrial Engineering, The University of Arizona, 1127 E. James E. Rogers Way, 85721, Tucson, AZ (USA).*

^b*Department of Mechanical and Aerospace Engineering, Sapienza University of Rome, Via Eudossiana 18, 00184, Rome, Italy.*

Abstract

This paper focuses on the application of meta-reinforcement learning to the robust design of low-thrust interplanetary trajectories in the presence of multiple uncertainties. A closed-loop control policy is used to optimally steer the spacecraft to a final target state despite the considered perturbations. The control policy is approximated by a deep recurrent neural network, trained by policy-gradient reinforcement learning on a collection of environments featuring mixed sources of uncertainty, namely dynamic uncertainty and control execution errors. The recurrent network is able to build an internal representation of the distribution of environments, thus better adapting the control to the different stochastic scenarios. The results in terms of optimality, constraint handling, and robustness on a fuel-optimal low-thrust transfer between Earth and Mars are compared with those obtained via a traditional reinforcement learning approach based on a feed-forward neural network.

Keywords: Meta-Reinforcement Learning, Robust Trajectory Design, Closed-Loop Guidance, Recurrent Neural Network, Proximal Policy Optimization, Stochastic Optimal Control

1. Introduction

Traditional optimal control methods, such as indirect methods based on Pontryagin maximum principle [1] or direct methods based on either collocation [2] or shooting [3], represent consolidated tools to plan optimal space trajectories. These methods are particularly effective when the trajectory design process is realized in a deterministic reference scenario. However, in real-world space missions, the spacecraft motion is inevitably affected by several sources of uncertainty, which may cause the actual trajectory to deviate significantly from the nominal one. These uncertainties may arise, as an example, due to unmodeled dynamics, inaccuracies in the orbital determination process, or control execution errors, and are typically non-trivial to handle within an optimal control framework.

During mission design, engineers usually check the reference trajectory robustness a posteriori by performing a navigation analysis with different uncertainty realizations. The robustness of this nominal design is then improved through an iterative procedure, which consists of gradually adding propellant margins and/or enforcing coasting phases for trajectory correction maneuvers until the mission outcome becomes acceptable, even in the perturbed scenarios [4, 5]. However, this design approach is generally time-consuming and tends to bring over-conservative margins.

Several stochastic optimization procedures have been proposed by the scientific community to integrate robustness into the trajectory optimization process. A classical approach is dynamic programming (DP) [6], which makes use of Bellman's equation to find an optimal control policy for every possible state. However, this approach is impractical for most real-world applications, as it is subject to the so-called curse of dimensionality. Approximate solutions methods have been proposed, such as differential dynamic programming (DDP), which iteratively solves a locally-expanded Bellman's equation around a reference trajectory, updated at each iteration [7]. Nonetheless, being based on a local expansion, the method usually struggles to

*Corresponding Author.

Email addresses: lorenzof@arizona.edu (Lorenzo Federici), alessandro.zavoli@uniroma1.it (Alessandro Zavoli)

Manuscript presented at the 73rd International Astronautical Congress (IAC), Paris, France, 18-22 September 2022, paper code IAC-22,C1,IPB,8,x68600. Copyright by Dr. Lorenzo Federici.

deal with large perturbations. Belief-based transcription procedures for stochastic optimal control problems (SOCPs) have also been proposed for the robust design of space trajectories under stochastic and epistemic uncertainties [8, 9], also incorporating navigation analysis in the formulation [10, 11]. In these approaches, the possible realizations of the system state, which are unknown because of model uncertainty and/or partial observability, are replaced by the belief states, that is, a probability distribution over real states. The main advantage of such formulation is that the belief state is completely observable; on the other hand, the belief state space is much more complex, as there exists an infinite number of probability distributions over the states. Model predictive control (MPC) has recently emerged as a flexible computational method for robust aerospace guidance [12]. MPC consists of repeatedly solving an optimal control problem (OCP) with initial conditions updated at each step on the basis of real-time measurements. However, in the presence of general nonlinear dynamics and constraints, the effectiveness of this method is reduced, as the increased computational cost negatively impacts the guidance update frequency. An alternative solution strategy consists of steering the system from an initial probability distribution, or covariance, to a desired value, by optimizing both an open-loop control signal and a state-feedback policy. This approach, referred to as covariance control, has been recently applied to low-thrust interplanetary trajectory design [13], rocket landing [14, 15], and other aerospace problems. Nevertheless, how to use this methodology to tackle more general stochastic problems with bounded uncertainties or non-additive disturbances is still an open question.

Artificial intelligence (AI), and in particular deep learning (DL), proved to be an effective and time-efficient approach to the solution of control problems in different research areas, first of all, robotics [16]. In the last few years, interest in using deep learning techniques has grown rapidly in the aerospace field. In space guidance applications, a deep neural network (DNN) can be used as a parametric model to map observations of the spacecraft state (typically, its position and velocity) to corresponding control actions, which define the magnitude and direction of the thrust. Reinforcement learning (RL) is one of the most used and successful training approaches. In RL, training data are directly collected through repeated interactions with the “environment”, which consists of simulations of the considered mission scenario, usually formulated as a Markov decision process (MDP). The only external feedback is a numerical reward provided at each simulation step, representing a

measure of the current network performance. Through the interaction with realizations of the environment, information is collected in the form of observation-control-reward tuples, which allows for more thoughtful decisions in successive training episodes. The control policy is progressively refined by maximizing the average cumulative reward over a single trajectory by stochastic gradient-based methods. Several research papers have already dealt with the use of deep reinforcement learning (DRL) for the robust closed-loop guidance of spacecraft during planetary landing maneuvers [17] and proximity operations [18], as well as for the design of cislunar [19] and interplanetary trajectories [20]. The robust design of an interplanetary mission has also been already addressed via DRL by considering the moderate presence of safe mode events [21] or navigation uncertainties [22].

As opposed to traditional solution methods, DRL algorithms can be straightforwardly applied to stochastic control problems with arbitrary transition probabilities and observation models, even provided in the form of black-box functions, since an explicit mathematical formulation is not required. Also, the exploratory behavior typical of RL algorithms provides inherent robustness against model uncertainty. Furthermore, the computational effort is almost completely concentrated in the network’s training phase, which is performed pre-flight on high-performing computing hardware. Conversely, the onboard computational time is just determined by one forward pass of the network per guidance step, which is extremely fast.

When DRL algorithms are combined with a recurrent architecture of the network, an advanced framework, commonly referred to as meta-reinforcement learning (meta-RL) or “learning to learn” [23], is obtained. Recurrent neural networks (RNNs), thanks to feedback connections, store information about the temporal evolution of the processed data into internal states. This capability allows them to better specialize the network output to the specific problem realization and significantly boosts their performance when dealing with non-Markov, partially observable, or multiple-task environments. An RNN-based meta-RL approach has already been exploited for the onboard guidance of a spacecraft or a lander in environments encompassing uncertain dynamics [24, 25], partial observability [26, 27, 28], or different tasks [29], always outperforming standard RL procedures based on a fully-connected network as control policy.

In this respect, this paper investigates the use of meta-reinforcement learning for the robust design of a low-thrust Earth-Mars transfer and for the associated closed-

loop guidance under the concurrent presence of uncertainties on the dynamic model and control execution errors. The specific study case of the paper has been selected as it represents a traditional benchmark for space trajectory optimization algorithms within the astrodynamics community [30, 31], and it allows for a thorough comparison against alternative robust trajectory design techniques [7, 13]. This work significantly extends a prior work by the same authors, which considered the different sources of uncertainty separately, did not consider any uncertainty in the spacecraft mass, and limited the analysis to the use of a standard RL procedure based on a feed-forward neural network controller [32]. Instead, in this manuscript, a meta-RL approach is implemented by training an RNN on a set of randomly generated environments featuring all the considered uncertainties. The objective is to obtain a flexible control policy able to adapt to different scenarios and produce high-quality results when facing different kinds of uncertainties together. The results in terms of optimality and robustness are compared with those obtained by a traditional RL procedure.

2. Problem Statement

This section presents the low-thrust interplanetary trajectory optimization problem considered as a study case by first describing a deterministic dynamic model and formulating it as a Markov decision process (MDP). Then, the uncertainties affecting the spacecraft motion are introduced, and two new formulations as stochastic MDPs are proposed by considering the effect of a single source of uncertainty at a time.

2.1. Markov Decision Process

When tackling SOCPs with RL, the problem is first reformulated as an equivalent discrete-time problem defined as Markov decision process (MDP). In an MDP, at each time step h , $h = 0, 1, \dots$, a decision maker (referred to as the agent) chooses an action or control $\mathbf{u}_h$ among the admissible ones, on the basis of an observation $\mathbf{y}_h$ of the current system (or environment) state $\mathbf{x}_h$, according to a closed-loop control policy π : $\mathbf{u}_h = \pi(\mathbf{y}_h)$. As a consequence of this action, the environment transitions to a new state $\mathbf{x}_{h+1}$ and returns a scalar reward $R_h = R(\mathbf{x}_h, \mathbf{u}_h, \mathbf{x}_{h+1})$, which can be intended as a measure of the desirability of the last state transition. MDPs satisfy the Markov property, that is, the next system state depends only on the previous state and the last agent's action. The goal in an MDP is to find the control policy π^* that maximizes the expected

value of the discounted return received along a trajectory $\tau = \{(\mathbf{x}_0, \mathbf{u}_0), (\mathbf{x}_1, \mathbf{u}_1), \dots\}$

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} [G(\tau)] \quad (1)$$

with

$$G(\tau) = \sum_{h=0,1,\dots} \gamma^h R_h \quad (2)$$

and $\gamma \in (0, 1]$ a discount factor on future rewards. Here, $\mathbb{E}_{\tau \sim \pi} [\cdot]$ indicates the expected value over several trajectories τ collected by interacting with the environment (i.e., the MDP) according to policy π .

When dealing with MDPs, two functions closely related to the trajectory return, which are useful for the derivation of solution procedures, are the value function V^π and action-value function Q^π , defined as

$$V^\pi(\mathbf{x}) = \mathbb{E}_{\tau \sim \pi} [G(\tau) | \mathbf{x}_0 = \mathbf{x}] \quad (3)$$

$$Q^\pi(\mathbf{x}, \mathbf{u}) = \mathbb{E}_{\tau \sim \pi} [G(\tau) | \mathbf{x}_0 = \mathbf{x}, \mathbf{u}_0 = \mathbf{u}] \quad (4)$$

2.2. Deterministic Model

Let us consider a spacecraft equipped with a low-thrust engine with maximum thrust $T_{\max}$ and effective exhaust velocity c . The spacecraft starts its interplanetary mission by leaving a planetary body p_1 at a pre-determined departure epoch $t_0 = 0$, with zero excess of hyperbolic velocity. Its goal is to reach a second planetary body p_2 with zero relative velocity at a prescribed arrival epoch t_f with minimum propellant expenditure.

2.2.1. Optimal Control Problem Formulation

By approximating the spacecraft as a point mass object, its state at any time $t \in [t_0, t_f]$ is completely described by its inertial position $\mathbf{r} = [x \ y \ z]^\top$ and velocity $\mathbf{v} = [v_x \ v_y \ v_z]^\top$ with respect to Sun and by its mass m

$$\mathbf{x} = [\mathbf{r}^\top \ \mathbf{v}^\top \ m]^\top \quad (5)$$

During its interplanetary journey, the spacecraft is subject both to the inverse-square gravity field of the primary body (the Sun) and to the engine thrust $\mathbf{T}$. The gravitational influence of the two planetary bodies p_1 and p_2 is instead neglected. The equations of motion of the spacecraft can be written as

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{T}) = \begin{bmatrix} \mathbf{v} \\ -\frac{\mu_\odot}{\|\mathbf{r}\|^3} \mathbf{r} + \frac{\mathbf{T}}{m} \\ -\frac{\|\mathbf{T}\|}{c} \end{bmatrix} \quad (6)$$

being $\mu_\odot$ the gravitational constant of the Sun. Let us use the subscripts 0 and f to indicate the value assumed

by physical quantities at time t_0 and t_f , respectively. The initial position and velocity of the spacecraft are the same as the departure planet p_1 , that is, $\mathbf{r}_{p_1,0}$ and $\mathbf{v}_{p_1,0}$. The terminal position and velocity must match those of the arrival planet p_2 , that is, $\mathbf{r}_{p_2,f}$ and $\mathbf{v}_{p_2,f}$.

Thus, the problem can be formulated as an OCP as follows

$$\mathcal{O} : \begin{cases} \max_T m_f \\ \text{s.t. } \dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{T}), & t \in [0, t_f] \\ \|\mathbf{T}\| \leq T_{\max}, & t \in [0, t_f] \\ \mathbf{r}_0 = \mathbf{r}_{p_1,0} \\ \mathbf{v}_0 = \mathbf{v}_{p_1,0} \\ \mathbf{r}_f = \mathbf{r}_{p_2,f} \\ \mathbf{v}_f = \mathbf{v}_{p_2,f} \end{cases} \quad (7)$$

Since the OCP in Eq. (7) is a non-convex optimization problem due to the non-convexity of the constraints on the spacecraft dynamics and on the maximum thrust, it possesses, in general, multiple locally optimal solutions.

2.2.2. Markov Decision Process Formulation

A possible way to formulate the OCP in Eq. (7) as an (almost) equivalent MDP is by approximating the low-thrust trajectory as a series of H ballistic arcs, with equal time length $\Delta t = t_f/H$, connected by impulsive $\Delta \mathbf{v}$ s, in a similar way as in the well-known Sims-Flanagan model [33]. This approach has already been used by Lantoine and Russell [30], who first introduced this benchmark trajectory optimization problem and solved it via DDP.

The spacecraft state $\mathbf{x}_h^-$ at any time t_h , $h = 0, \dots, H$, before the impulse (superscript “-”), is identified by its inertial position $\mathbf{r}_h = [x_h \ y_h \ z_h]^\top$ and velocity $\mathbf{v}_h^- = [v_{x,h}^- \ v_{y,h}^- \ v_{z,h}^-]^\top$ with respect to Sun, by its mass m_h^- , and by the current time t_h itself

$$\mathbf{x}_h^- = [\mathbf{r}_h^\top \ (\mathbf{v}_h^-)^\top \ m_h^- \ t_h]^\top \in \mathbb{R}^7 \times [0, t_f] \quad (8)$$

The magnitude of the impulse $\Delta \mathbf{v}_h$ performed at time t_h is limited by the amount of velocity variation that could be realized by operating the engine at maximum thrust $T_{\max}$ for a time Δt

$$\Delta v_{\max,h} = \frac{T_{\max}}{m_h^-} \Delta t \quad (9)$$

The control $\mathbf{u}_h = [u_h^0 \ \tilde{\mathbf{u}}_h^\top]^\top \in [-1, 1]^4$ returned by policy π at step h , $h = 0, \dots, H-1$, defines the impulsive $\Delta \mathbf{v}$ maneuver. Specifically, the term u_h^0 impacts the magnitude of the velocity variation, while the vector $\tilde{\mathbf{u}}_h$ its

direction in the Sun-centered inertial reference frame:

$$\Delta \mathbf{v}_h = \mathbf{v}(\mathbf{u}_h) = \Delta v_{\max,h} \frac{1 + u_h^0}{2} \frac{\tilde{\mathbf{u}}_h}{\|\tilde{\mathbf{u}}_h\|} \quad (10)$$

It is worth noting that this definition of the control inherently meets the constraint on the maximum value of the impulsive velocity variation (Eq. (9)).

The terminal $\Delta \mathbf{v}$, which is performed at time $t_f = t_H$, is not the output of policy π , but it is computed algebraically in order to match the destination planet velocity unless this would lead to a violation of the constraint in Eq. (9), that is

$$\Delta \mathbf{v}_H = \begin{cases} -\mathbf{v}_{\infty,H} & \|\mathbf{v}_{\infty,H}\| < \Delta v_{\max,H} \\ \Delta v_{\max,H} \hat{\mathbf{v}}_{\infty,H} & \text{otherwise} \end{cases} \quad (11)$$

with $\mathbf{v}_{\infty,H} = \mathbf{v}_H^- - \mathbf{v}_{p_2,f}$

Since the spacecraft moves under Keplerian dynamics between any two impulses, in a deterministic scenario, the spacecraft state can be propagated analytically with a closed-form transition function ϕ

$$\mathbf{x}_{h+1}^- = \phi(\mathbf{x}_h^+) = \begin{bmatrix} \tilde{f}_h \mathbf{r}_h + \tilde{g}_h \mathbf{v}_h^+ \\ \tilde{f}_h \mathbf{r}_h + \tilde{g}_h \mathbf{v}_h^+ \\ m_h^+ \\ t_h + \Delta t \end{bmatrix} \quad (12)$$

$$\mathbf{x}_h^+ = \zeta(\mathbf{x}_h^-, \Delta \mathbf{v}_h) = \begin{bmatrix} \mathbf{r}_h \\ \mathbf{v}_h^- + \Delta \mathbf{v}_h \\ m_h^- \exp(-\|\Delta \mathbf{v}_h\|/c) \\ t_h \end{bmatrix} \quad (13)$$

where $\tilde{f}_h$ and $\tilde{g}_h$ are the Lagrange’s coefficients at h -th step, defined as in Ref. [34].

The goal of the optimization procedure is to maximize the final mass of the spacecraft while ensuring compliance, up to a given accuracy ε , with terminal rendezvous constraints on position and velocity. So, the reward R_h obtained at the end of step h has been defined as

$$R_h = \begin{cases} \Delta m_h & h < H \\ \Delta m_h - \lambda_e \max(0, e_{x,h+1} - \varepsilon) & h = H \end{cases} \quad (14)$$

where $\Delta m_h = m_h^+ - m_h^-$ is the (non-dimensional) consumed propellant mass at time t_h and $e_{x,H}$ is the maximum value between the final position error $e_r = \|\mathbf{r}_H - \mathbf{r}_{p_2,f}\|/\|\mathbf{r}_{p_2,f}\|$ and velocity error $e_v = \|\mathbf{v}_H^+ - \mathbf{v}_{p_2,f}\|/\|\mathbf{v}_{p_2,f}\|$. In the reward function, the difference in mass Δm_h at each step has been used in place of the final spacecraft mass m_f at the end of the episode since, whenever possible, a frequent reward should be

preferred within an RL framework as it gives the agent an immediate feedback about the quality of the last action taken. The same approach cannot be used for the terminal constraint violations, as they can be only evaluated at the final step of each episode and thus must be included in the reward function as delayed terms.

The value selected for the tolerance ε on terminal constraint violation is discussed in Sec. 5.2. The penalty weight $\lambda_e = 50$ is used in this application. Although this penalty weight does not exert a significant influence during the training process, thanks to the ε -constraint approach adopted in this study (see Sec. 5.2), its value becomes important in the final training iterations to effectively balance the propellant mass and terminal constraint violation components within the reward function. As a result, a value of $\lambda_e = 50$ has been chosen to ensure that both terms possess a similar order of magnitude.

So, the MDP formulation of the deterministic problem is:

$$M_{\text{unp}} : \begin{cases} \max_{\pi} G(\tau) \\ \text{s.t. } \mathbf{x}_{h+1}^- = \boldsymbol{\phi}(\mathbf{x}_h^+), & h \in [0, H) \\ \mathbf{x}_h^+ = \boldsymbol{\zeta}(\mathbf{x}_h^-, \Delta \mathbf{v}_h), & h \in [0, H) \\ \Delta \mathbf{v}_h = \mathbf{v}(\mathbf{u}_h), & h \in [0, H) \\ \mathbf{u}_h = \boldsymbol{\pi}(\mathbf{y}_h), & h \in [0, H) \\ \mathbf{y}_h = \mathbf{x}_h^-, & h \in [0, H) \\ \mathbf{x}_0^- = [\mathbf{r}_{p1,0}^T \ \mathbf{v}_{p1,0}^T \ m_0 \ t_0]^T, \\ t_H = t_f \end{cases} \quad (15)$$

2.3. Stochastic Model

Let us now introduce the probability models used to describe the different sources of uncertainty considered in this study.

2.3.1. Dynamic Uncertainties

The effect of uncertainties on the spacecraft dynamics, that is, unmodeled accelerations and/or inaccurate model parameters, is modeled as an additive Gaussian noise on position, velocity, and mass at time t_h , $h = 0, \dots, H-1$, that is, as a disturbance

$$\mathbf{w}_{\text{st},h} = \begin{bmatrix} \delta \mathbf{r}_h \\ \delta \mathbf{v}_h^- \\ \delta m_h^- \\ \delta t_h \end{bmatrix} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}_{\text{st}}) \in \mathbb{R}^8 \quad (16)$$

where $\boldsymbol{\Sigma}_{\text{st}} = \text{diag}(\sigma_{r,\text{st}}^2 \mathbf{I}_3, \sigma_{v,\text{st}}^2 \mathbf{I}_3, \sigma_{m,\text{st}}^2, 0)$ is the covariance matrix, with $\sigma_{r,\text{st}}$, $\sigma_{v,\text{st}}$, and $\sigma_{m,\text{st}}$ the standard deviations on position, velocity and mass, and where $\mathbf{I}_n$

indicates an $n \times n$ identity matrix. $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, instead, indicates a multi-variate normal distribution with mean $\boldsymbol{\mu}$ and covariance $\boldsymbol{\Sigma}$.

Thus, in the presence of dynamic uncertainties, the stochastic dynamic model of the spacecraft is

$$\mathbf{x}_{h+1}^- = \boldsymbol{\phi}(\mathbf{x}_h^+) \quad (17)$$

$$\begin{aligned} \mathbf{x}_h^+ &= \boldsymbol{\eta}_{\text{st}}(\mathbf{x}_h^-, \Delta \mathbf{v}_h, \mathbf{w}_{\text{st},h}) \\ &= \boldsymbol{\zeta}(\mathbf{x}_h^-, \mathbf{w}_{\text{st},h}, \Delta \mathbf{v}_h) \end{aligned} \quad (18)$$

2.3.2. Control Errors

Errors in the execution of the commanded control are modeled as a random three-dimensional rotation of the $\Delta \mathbf{v}$, defined by Euler angles $(\delta\varphi, \delta\vartheta, \delta\psi)$, and a random variation δu of its modulus. Random variables $\delta\varphi, \delta\vartheta, \delta\psi$ and δu are sampled at each time t_h from Gaussian distributions with standard deviations $\sigma_\varphi, \sigma_\vartheta, \sigma_\psi$ and σ_u , respectively. So, the disturbance on control at time t_h is

$$\mathbf{w}_{\text{ctr},h} = \begin{bmatrix} \delta\varphi_h \\ \delta\vartheta_h \\ \delta\psi_h \\ \delta u_h \end{bmatrix} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}_{\text{ctr}}) \in \mathbb{R}^4 \quad (19)$$

where $\boldsymbol{\Sigma}_{\text{ctr}} = \text{diag}(\sigma_\varphi^2, \sigma_\vartheta^2, \sigma_\psi^2, \sigma_u^2)$ is the covariance matrix. The sampled value of δu is properly clipped so that the corresponding impulse does not exceed the maximum value.

The actual impulse $\Delta \tilde{\mathbf{v}}_h$ can be written as a function of the commanded impulse $\Delta \mathbf{v}_h$ at time t_h , $h = 0, \dots, H-1$, as

$$\Delta \tilde{\mathbf{v}}_h = \mathbf{A}(\mathbf{w}_{\text{ctr},h}) \Delta \mathbf{v}_h \quad (20)$$

where the rotation-amplification matrix $\mathbf{A}$ is evaluated, under the small-angle assumption, as

$$\mathbf{A}(\mathbf{w}_{\text{ctr},h}) = (1 + \delta u_h) \begin{bmatrix} 1 & -\delta\psi_h & \delta\vartheta_h \\ \delta\psi_h & 1 & -\delta\varphi_h \\ -\delta\vartheta_h & \delta\varphi_h & 1 \end{bmatrix} \quad (21)$$

Thus, in the presence of control errors, the stochastic dynamic model of the spacecraft is

$$\mathbf{x}_{h+1}^- = \boldsymbol{\phi}(\mathbf{x}_h^+) \quad (22)$$

$$\begin{aligned} \mathbf{x}_h^+ &= \boldsymbol{\eta}_{\text{ctr}}(\mathbf{x}_h^-, \Delta \mathbf{v}_h, \mathbf{w}_{\text{ctr},h}) \\ &= \boldsymbol{\zeta}(\mathbf{x}_h^-, \mathbf{A}(\mathbf{w}_{\text{ctr},h}) \Delta \mathbf{v}_h) \end{aligned} \quad (23)$$

It is worth noticing that this disturbance cannot be modeled as an additive Gaussian noise on the state, and most of the traditional solution techniques for SOCPs in the literature cannot be directly applied.

Summing up, the MDP in Eq. (15) gives rise to two new stochastic decision processes, M_a , $a \in \{\text{st}, \text{ctr}\}$, depending on what uncertainty is considered:

$$M_a : \begin{cases} \max_{\pi} \mathbb{E}_{\tau \sim \pi} [G(\tau)] \\ \text{s.t. } \mathbf{x}_{h+1}^- = \phi(\mathbf{x}_h^+), & h \in [0, H) \\ \mathbf{x}_h^+ = \eta_a(\mathbf{x}_h^-, \Delta \mathbf{v}_h, \mathbf{w}_{a,h}), & h \in [0, H) \\ \Delta \mathbf{v}_h = \mathbf{v}(\mathbf{u}_h), & h \in [0, H) \\ \mathbf{u}_h = \pi(\mathbf{y}_h), & h \in [0, H) \\ \mathbf{y}_h = \mathbf{x}_h^-, & h \in [0, H) \\ \mathbf{w}_{a,h} \sim \mathcal{N}(\mathbf{0}, \Sigma_a), & h \in [0, H) \\ \mathbf{x}_0^- = [\mathbf{r}_{p1,0}^T \ \mathbf{v}_{p1,0}^T \ m_0 \ t_0]^T, \\ t_H = t_f \end{cases} \quad (24)$$

3. Deep Reinforcement Learning

Reinforcement Learning is the area of artificial intelligence concerned with how an agent (i.e., a computer program) should behave, that is, map observations to actions through a policy π , to maximize the expected trajectory return. Learning occurs through repeated interactions with the environment, modeled as an MDP, and by using the collected observation–action–reward tuples to progressively refine the control policy π by gradient-based optimization.

3.1. Policy Gradient Methods

In policy-gradient DRL methods, the agent directly learns an approximation $\hat{\pi}$ of the optimal control policy π^* , provided by a deep neural network (DNN) π_θ , generally a feed-forward network as a multi-layer perceptron (MLP), with parameters (i.e., weights and biases) θ . The approximation is updated by directly maximizing the trajectory return J , or alternative objective functions still related to J but featuring additional regularization terms to improve the convergence properties of the training procedure. In policy-gradient RL, a stochastic policy (typically, a diagonal multi-variate Gaussian) is used during training to enhance the exploration of the solution space. Thus, given as input the current observation $\mathbf{y}$, the network returns the mean value $\boldsymbol{\mu}_\theta$ and covariance $\boldsymbol{\sigma}_\theta$ of the distribution. The covariance may be a standard output of the network, thus dependent on the input, or can be treated as an additional bias of the network, thus just dependent on the network parameters. Therefore, the returned control is computed as

$$\pi(\mathbf{y}) = \begin{cases} \mathcal{N}(\boldsymbol{\mu}_\theta(\mathbf{y}), \Sigma_\theta(\mathbf{y})) & \text{in training} \\ \boldsymbol{\mu}_\theta(\mathbf{y}) & \text{in evaluation or deployment} \end{cases} \quad (25)$$

with $\Sigma_\theta = \text{diag}(\sigma_\theta)^2$.

In some policy-gradient algorithms, named actor-critic, a second DNN (the critic), V_ϕ , with parameters ϕ , is used to approximate the value function V^π , and trained concurrently to the policy network π_θ (or actor). Alternatively, a single shared network architecture, with parameters θ , can be used to compute both the control policy π_θ and the value function V_θ to speed up the training procedure.

3.2. Proximal Policy Optimization

The training algorithm used in this study is proximal policy optimization (PPO) [35]. PPO is an actor-critic method conceived in 2017 by Schulman et al. at OpenAI [35] as an unconstrained, first-order trust region method. The objective function of PPO is devised to take the largest step possible in the network’s parameter space without causing a performance collapse. This is achieved by clipping to zero the probability that the updated policy π_θ differs more than ϵ percent from the policy $\pi_{\theta(k-1)}$ at the previous training iteration $k-1$

$$J^{\text{clip}}(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\frac{1}{H} \sum_{h=0}^{H-1} \min \left\{ \tilde{r}_h(\theta) A^{\pi_\theta}(\mathbf{y}_h, \mathbf{u}_h), \right. \right. \\ \left. \left. \text{clip}(\tilde{r}_h(\theta), 1 - \epsilon, 1 + \epsilon) A^{\pi_{\theta(k-1)}}(\mathbf{y}_h, \mathbf{u}_h) \right\} \right] \quad (26)$$

where

$$\tilde{r}_h(\theta) = \frac{\pi_\theta(\mathbf{u}_h | \mathbf{y}_h)}{\pi_{\theta(k-1)}(\mathbf{u}_h | \mathbf{y}_h)} \quad (27)$$

$$A^{\pi_\theta}(\mathbf{y}_h, \mathbf{u}_h) = Q^{\pi_\theta}(\mathbf{y}_h, \mathbf{u}_h) - V^{\pi_\theta}(\mathbf{y}_h) \quad (28)$$

are the probability ratio between the new and old policy and the advantage function, respectively, and the clip function is defined as follows

$$\text{clip}(\tilde{r}_h(\theta), 1 - \epsilon, 1 + \epsilon) = \begin{cases} 1 - \epsilon & \tilde{r}_h(\theta) \leq 1 - \epsilon \\ 1 + \epsilon & \tilde{r}_h(\theta) \geq 1 + \epsilon \\ \tilde{r}_h(\theta) & \text{otherwise} \end{cases} \quad (29)$$

The clip range ϵ is a fundamental hyperparameter of the method, as it removes incentives for the policy to change dramatically.

PPO is an actor-critic method as it also needs an estimate V_ϕ of the value function V^{π_θ} to approximate the advantage A^{π_θ} via the generalized advantage estimator (GAE) [36]

$$A_{\phi,h} = \sum_{h'=h}^{H-1} (\gamma \lambda)^{h'-h} \delta_{h'}^{V_\phi} \quad (30)$$

where

$$\delta_h^{V_\phi} = R_h + \gamma V_{\phi,h+1} - V_{\phi,h} \quad (31)$$

is the temporal-difference residual of $V_{\phi,h}$ with discount γ , and where $\lambda \in [0, 1]$ is defined GAE factor. The value function estimate is updated by minimizing the mean-squared-error (MSE) $L^{\text{value}}(\phi)$ between its value and the reward-to-go, that is, the cumulative reward obtained from step h onwards.

$$L^{\text{value}}(\theta, \phi) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\frac{1}{H} \sum_{h=0}^{H-1} \left(V_{\phi,h} - \sum_{h'=h}^H R_{h'} \right)^2 \right] \quad (32)$$

When a single shared agent network is used (playing both the actor and critic role), the parameter set is unique, $\theta = \theta \cup \phi$, and the MSE is added to the clipped policy objective function to create the final PPO objective

$$J^{\text{ppo}}(\theta) = J^{\text{clip}}(\theta) - \lambda_v L^{\text{value}}(\theta) \quad (33)$$

with λ_v known as value function coefficient.

The training process by PPO works as follows. At each iteration, the most up-to-date policy is run in n_{env} parallel environments to collect a total of n_t trajectories. These trajectories are randomly split into n_b mini-batches and used to compute an estimate of the gradient of the PPO objective with an empirical average. Then, the new network's parameters are computed via n_{opt} stochastic gradient optimization steps. When a shared actor-critic agent network is used, the parameters θ are updated all at once by maximizing the PPO objective

$$\theta_{(k+1)} = \theta_{(k)} + \alpha_{(k)} \nabla J^{\text{ppo}}(\theta)|_{\theta_{(k)}} \quad (34)$$

with $\alpha_{(k)}$ the learning rate at iteration k . Conversely, with independent actor and critic networks, the parameters θ and ϕ are updated separately

$$\theta_{(k+1)} = \theta_{(k)} + \alpha_{(k)} \nabla_\theta J^{\text{clip}}(\theta, \phi)|_{\theta_{(k)}, \phi_{(k)}} \quad (35)$$

$$\phi_{(k+1)} = \phi_{(k)} - \alpha_{(k)} \nabla_\phi L^{\text{value}}(\theta, \phi)|_{\theta_{(k)}, \phi_{(k)}} \quad (36)$$

During training, a deterministic version of the current policy is concurrently evaluated for n_e episodes in one separate environment for validation purposes. The training procedure is stopped after K training iterations.

4. Meta-Reinforcement Learning

Standard MLP networks have been widely used as control policies in DRL to solve MDPs over both discrete and continuous state and action spaces. However, when a single MLP is asked to solve multiple, yet somehow related, problems, which may, for example, differ

just for the transition, observation, or reward function, it typically struggles to correctly specialize the output on the basis of the considered task. In these cases, a network comprised of a very large number of parameters is, in principle, required, thus greatly complicating the training procedure by RL. Furthermore, MLPs have no way to deal with all those problems where the optimal control does not depend just on the last observation but also on the preceding ones, such as non-Markov or partially observable decision processes.

In these cases, meta-reinforcement learning (meta-RL) [37] may significantly improve the neural network capability of reconstructing the optimal control policy. The term meta-learning, or “learning-to-learn”, refers to a family of machine learning procedures whose objective is to autonomously learn a learning procedure to correctly cope with several, and slightly different, tasks T , for example, sampled from a distribution $\mathcal{P}(T)$. Each task is described by its own dataset D_T and objective function J_T . In reinforcement learning, each task T may represent a different MDP (as in this study), with its own collection of trajectories $D_T = \{\tau_i\}_{i=0,1,\dots}$ and trajectory return $G_T(\tau) = \sum_{h=0,1,\dots} \gamma^h R_{T,h}$. The objective of meta-RL is to learn the network parameters θ in such a way that $\pi(y; \mathbf{g}_\theta(T))$ is the optimal control policy for task T , with $\mathbf{g}_\theta(T)$ a function of the specific task. The parameters θ are computed to maximize the expected value of the return G_T over the different tasks

$$\max_{\theta} \mathbb{E}_{T \sim \mathcal{P}(T)} \left[\mathbb{E}_{\tau \sim \pi(\cdot; \mathbf{g}_\theta(T))} [G_T(\tau)] \right] \quad (37)$$

One of the possible approaches to implement meta-RL is to include one or more recurrent layers inside the policy network [23]. Recurrent neural networks (RNN) are a family of NNs designed for processing sequential input data and keeping information about their temporal variation, thanks to a feedback connection. If an RNN is used as a policy network, the output at step h , denoted as $\mathbf{h}_h$, is dependent, through the network parameters θ and activation function, both on the current input y_h and on the previous output $\mathbf{h}_{h-1}$, also defined hidden network state, as schematized in Fig. 1.

When the RNN is trained to perform a task that requires remembering past information, the network learns to use $\mathbf{h}_h$ as a kind of “summary” of the most relevant features of the past sequence of input up to step h . Thus, the hidden state $\mathbf{h}$ is what allows the information about previous time points to persist. This characteristic makes the policy network able to deal both with partially observable and non-Markov processes and with multiple tasks, as now the network's output is depen-

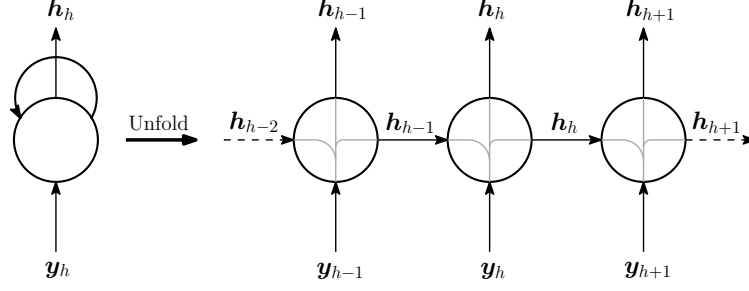


Figure 1: Unrolled computational graph of a RNN.

dent on the history of observations (which, in general, is specific to the considered task).

The network’s output can be further differentiated between the different tasks by providing as input to the recurrent network also the previous control $\mathbf{u}_{h-1}$ and corresponding reward R_h [38]. In this case, the task-specific function $\mathbf{g}_\theta(\mathbf{T})$ consists in the hidden network state: $\pi(\mathbf{y}; \mathbf{g}_\theta(\mathbf{T})) = \pi_\theta(\mathbf{y}; \mathbf{h}_\theta(\mathbf{T}))$.

A number of works in the literature already proved that this approach to meta-RL can consistently enhance the network performance when facing multiple space guidance problems [29] and partially-observable mission scenarios [27]. In the case study considered in the present manuscript, the distinct tasks correspond to the various MDPs in Eqs. (15) and (24). Each MDP features a distinct uncertainty distribution. Consequently, the meta-RL approach employing RNNs can be efficiently exploited to learn how to cope correctly with different kinds of uncertainties. The goal is to leverage the demonstrated generalization capabilities of RNNs to determine a control policy capable of effectively addressing all of the considered uncertainties concurrently within a single environment.

Thus, the main original element of this research work, with respect to previous studies in the literature about robust interplanetary trajectory design via RL [21, 32, 22], is the adoption, for the first time, of a meta-RL approach to train the control policy on different uncertainties distributions. This is implemented by using a recurrent layer within the policy network and by feeding it, at each step, with the whole set of preceding observations, control actions, and rewards instead of just the last observation received. The recurrent network is trained via standard PPO using a back-propagation through time algorithm [39] during the SGA-based update of the network parameters.

5. Numerical Results

A time-fixed, fuel-optimal transfer between Earth and Mars is used as a case study for this paper for the sake of comparison with other studies on robust trajectory design in the literature [7, 13]. The mission data are reported in Table 1. All physical quantities have been made non-dimensional by using as reference values the Earth-Sun mean distance $\bar{r} = 149.6 \times 10^6$ km, the Earth’s circular velocity $\bar{v} = \sqrt{\mu_\odot / \bar{r}}$, and the initial spacecraft mass $\bar{m} = m_0$. The number of time steps per episode H is equal to 40, as in the original paper by Lantoine and Russell [30]. The parameters defining the uncertainty distributions considered in the study are listed in Table 2.

Table 1: Problem data.

Variable	Value
t_f , days	348.79
m_0 , kg	1000
$T_{\max}$, N	0.50
c , km/s	19.6133
$\mu_\odot$, km ³ /s ²	132712440018
$\mathbf{r}_{p1,0}$, km	[−140699693, −51614428, 980]
$\mathbf{v}_{p1,0}$, km/s	[9.7746, −28.0783, $4.3377 \cdot 10^{-4}$]
$\mathbf{r}_{p2,f}$, km	[−172682023, 176959469, 7948912]
$\mathbf{v}_{p2,f}$, km/s	[−16.4274, −14.8605, $9.2149 \cdot 10^{-2}$]

During the network’s training, each of the n_{env} parallel training environments is one among the three MDPs $\mathbf{M}_{\text{unp}}$, $\mathbf{M}_{\text{st}}$, and $\mathbf{M}_{\text{ctr}}$ defined in Eqs. (15) and (24), randomly sampled at each iteration. The evaluation environment, instead, is a single environment featuring all the uncertainties of the different MDPs (that is, both on the dynamics and control).

5.1. Policy Networks

Two different agent networks are considered in this study to compare meta-RL with standard RL. The first

Table 2: Parameters of the uncertainty models.

Uncertainty	Parameter	Value
dynamics	$\sigma_{r,st}$, km	1.0
	$\sigma_{v,st}$, km/s	0.05
	$\sigma_{m,st}$, kg	1.0
control	σ_φ , deg	1.0
	σ_θ , deg	1.0
	σ_ψ , deg	1.0
	σ_u	0.05

one, used to implement a standard RL procedure, is a fully connected actor-critic architecture, shown in Fig. 3a. Both the actor and the critic are 3-layer MLPs. For either network, the three hidden layers have size $[10n_i, 10\sqrt{n_i n_o}, 10n_o]$, where $n_i = 8$ is the number of inputs, and $n_o = 4$ is the size of the returned control. The activation function is tanh for the hidden layers and linear for the output layer. The covariance is not dependent on the specific input. This particular network architecture has already found applications in various robust space guidance problems through RL [17, 18], where it has been employed to map a vector of spacecraft state-related observations to the optimal thrust in a similar way as in this study. These applications include a previous work by the authors [32] focused on the same robust trajectory design problem. Across these studies, the policy network consistently showed the ability to find nearly optimal control laws and achieve the desired level of accuracy in constraint satisfaction in the presence of different sources of uncertainties, among which unmodeled dynamics, navigation inaccuracies, and control errors. Consequently, it has been selected in this work as a baseline for evaluating the performance of meta-RL. This first agent network will be referred to as π^{mlp} .

The second network, used to implement a meta-RL procedure and shown in Fig. 3b, features a single shared actor-critic architecture. It is made up of two MLP layers, with size $[5n_i, 5\sqrt{n_i(n_o + 1)}]$, followed by a recurrent layer with $5(n_o + 1)$ nodes. The recurrent layer takes as input the previous control and reward as well. The size of this second network has been decided so that the two agent networks have roughly the same number of parameters. Specifically, in this application, a long short-term memory (LSTM) layer has been used, as, differently from standard RNNs, it does not undergo a reduction in performance in the presence of long-term time dependencies in the input. The main difference between LSTMs and standard RNNs lies in a separation

between the network output $\mathbf{h}$ and the hidden network state $\mathbf{c}$. Within the LSTM cell, this state undergoes modifications through a series of mechanisms referred to as gates: an input gate, a forget gate, and an output gate. These gates govern the flow of data within the cell, determining what information to retain and what to discard from the state at the previous time step $\mathbf{c}_{h-1}$, based on the values of the preceding output $\mathbf{h}_{h-1}$ and of the current input $\mathbf{y}_h$. In turn, the value of the current network output is determined by combining together, through the network weights, the last input $\mathbf{y}_h$, the previous output $\mathbf{h}_{h-1}$ and the cell state $\mathbf{c}_h$. A schematic of the internal gates of an LSTM cell is shown in Fig. 2.

An interested reader can find more detailed information about LSTM units in Ref. [40]. This second agent network will be referred to as π^{lstm} to clearly highlight that the meta-RL approach utilized in this paper derives from the presence of the LSTM layer within the policy network itself.

5.2. Algorithm Settings

The numerical results presented in this section have been obtained by using pyRLprob¹, an in-house library, based on Ray RLlib [41], for training, evaluation, and post-processing of RL agents in OpenAI-Gym-based environments. A multiple-CPU implementation of PPO from RLlib has been used as the optimization algorithm.

In order to improve RL effectiveness in solving the constrained problems in Eq. (24), a constraint relaxation technique already used in metaheuristic optimization [42] and in similar RL applications [18, 29], named ε -constraint, has been adopted in this paper. When using the ε -constraint approach, the terminal constraints are gradually enforced during optimization by decreasing the value of the constraint satisfaction tolerance ε , in order to explore the solution space to a greater extent and achieve better values of the trajectory return. A piecewise constant trend with just two values has been used in this application, as it showed the best overall performance in a previous study on the same problem by the authors [32]:

$$\varepsilon_{(k)} = \begin{cases} 1 \times 10^{-2} & k \leq \frac{3}{7}K \\ 1 \times 10^{-3} & k > \frac{3}{7}K \end{cases} \quad (38)$$

where k is the current training iteration, and K is the total number of training iterations. The final values for the position and velocity tolerances were chosen to align with the precision of the dynamic model employed in

¹<https://github.com/LorenzoFederici/pyrlprob>

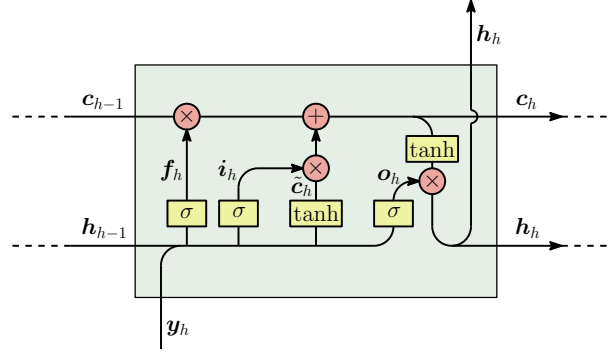
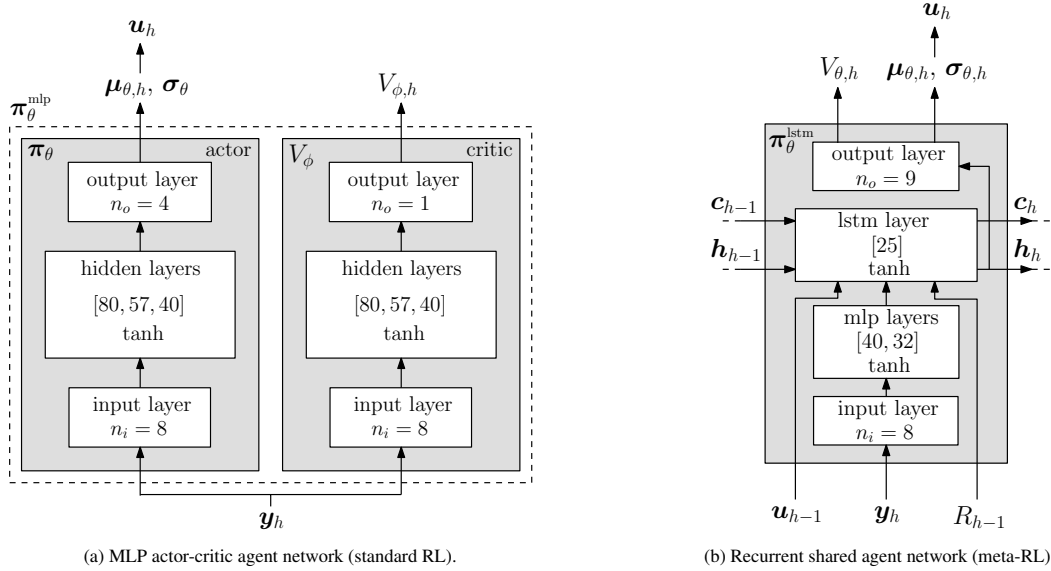


Figure 2: LSTM cell.



(a) MLP actor-critic agent network (standard RL).

(b) Recurrent shared agent network (meta-RL).

Figure 3: Architectures of the two agent networks compared in the paper.

this study. Indeed, as shown in Appendix 7, the impulsive maneuver approximation introduces an error of approximately $10^{-3}\bar{r}$ in position. The two-body model, on the other hand, results in even greater inaccuracies as it assumes negligible gravitational influence from the two planetary bodies. Notably, this model starts exhibiting significant deviations from the actual dynamic model when the spacecraft approaches the sphere of influence of the planetary body, which has a radius on the order of $10^{-2}\bar{r}$.

As explained in Section 3.2, when using a single shared policy network, the minimum squared error (MSE) on the value function and the clipped policy objective function must be summed together to create a single objective function for updating the network parameters (Eq. 33). Hence, it is crucial for both terms to have a comparable order of magnitude to prevent one of

them from predominantly influencing the optimization process. Given that the clipped policy objective is typically around 10^0 (as the policy π represents a probability), it follows that the MSE on the value function and, consequently, the cumulative reward should also be of a similar order of magnitude. Therefore, the absolute value of the reward, which initially fell in the range of 10^2 during the early training iterations, when it reached its peak, was divided by 100 to rescale it to approximately 1.

A manual trial-and-error approach was employed to fine-tune the primary hyperparameters of PPO to be used during the training process. This iterative process aimed to identify a suitable set of parameter values, and the resulting tuned parameters are listed in Table 3. The same set of hyperparameters was applied to both policy networks (standard RL and meta-RL). To maintain

stability during the latter stages of training, a linearly decreasing learning rate was utilized. Each network underwent training for a total of 140 million steps.

Table 3: PPO hyperparameters.

Hyperparameter	Symbol	Value
Training iterations	K	35000
Discount factor	γ	0.9999
GAE factor	λ	0.99
Clip range	ϵ	0.05
Value function coefficient	λ_v	0.5
Training environments	n_{env}	100
Training episodes per iter.	n_t	100
Evaluation episodes per iter.	n_e	100
Mini-batches	n_b	8
SGA iterations per update	n_{opt}	40
Initial learning rate	$\alpha_{(0)}$	1×10^{-4}
Final learning rate	$\alpha_{(N)}$	1×10^{-8}

5.3. Learning Curves

Figure 4 shows the trend along training of the mean value (solid line) and inter-quartile range (shaded region) of the trajectory return (Fig. 4a), the terminal error in the state (Fig. 4b), and the final spacecraft mass (Fig. 4c) obtained in the evaluation environment with policies π^{mlp} and π^{lstm} . It is apparent that, at the final training iterations, π^{lstm} outperforms network π^{mlp} both in terms of final spacecraft mass and terminal constraint violation, thus achieving a higher return on average. One can notice that all test metrics feature a sudden drop in performance at 3/7 of the total training time. This is a direct consequence of the reduction of the constraint tolerance ϵ in Eq. (38), which occurs exactly at that iteration.

5.4. Open-loop Performance Analysis

The open-loop performance of meta-RL and standard RL is first assessed by deploying, in the nominal mission scenario M_{unp} , the two best policies π^{mlp} and π^{lstm} obtained at the end of the training procedure. The two corresponding trajectories, shown in Fig. 5, represent robust trajectories against the considered dynamic and control uncertainties. For the sake of comparison, Fig. 5 also shows the deterministic optimal trajectory (in blue) computed with an indirect optimization method [43] by directly solving the OCP in Eq. (7). The differences between the robust trajectories and the optimal trajectory in Fig. 5 are exaggerated by a factor of 7 for illustration purposes. In both robust trajectories, and especially in

the one obtained with policy π^{lstm} , Mars orbit is reached (in position) slightly in advance compared to the optimal solution by the indirect approach. This behavior increases the chances of meeting the terminal rendezvous condition when uncertainties arise near the end of the mission, as the final burn executed by the policy primarily corrects the final velocity, given that the position is already very close to the desired location.

Table 4 summarizes the main open-loop performance parameters of the two agents, which include the final spacecraft mass m_f , the final position error e_r and velocity error e_v , and the trajectory return $G(\tau)$. For comparison, the results obtained with the indirect optimization approach in a deterministic scenario have also been included in the table. It is interesting to note that both networks satisfy the rendezvous condition with a similar margin on the desired tolerance (10^{-3}). With policy π^{lstm} , robustness against both dynamic and control uncertainties is obtained by sacrificing just 0.3% of the final spacecraft mass with respect to the optimal deterministic solution by the indirect approach. The final mass reduction is, instead, roughly ten times higher (3%) when policy π^{mlp} is used, thus confirming the improved performance of meta-RL. Indeed, as can be clearly seen in Fig. 7, the thrust trend along the robust trajectory (solid-line curve) returned by policy π^{lstm} more accurately resembles a bang-off-bang trend, which is well known to be the optimal one with dynamics affine in the control variables [1]. In both cases, the error in the final velocity is zero, as the last Δv is computed algebraically as a difference between the final spacecraft velocity and Mars velocity (see Eq. (11)).

It is essential to emphasize here that the worse results achieved through standard RL are unlikely to be attributed to an insufficient training duration or the utilization of suboptimal hyperparameters within the PPO algorithm. Indeed, as can be clearly observed in Fig. 4, the two learning curves obtained with standard RL and meta-RL gradually approach a stable value in the final training iterations. Therefore, extending the training time is unlikely to yield improved results for either policy. Additionally, as previously mentioned in Sec. 5.2, the PPO hyperparameters were preliminarily tuned and tested for both networks to determine the set that produces the best policy for each configuration. Consequently, it is also improbable that an alternative set of hyperparameters would result in superior outcomes for the MLP network.

5.5. Closed-loop Performance Analysis

The trained networks also provide the spacecraft with a computationally inexpensive, yet robust, closed-loop

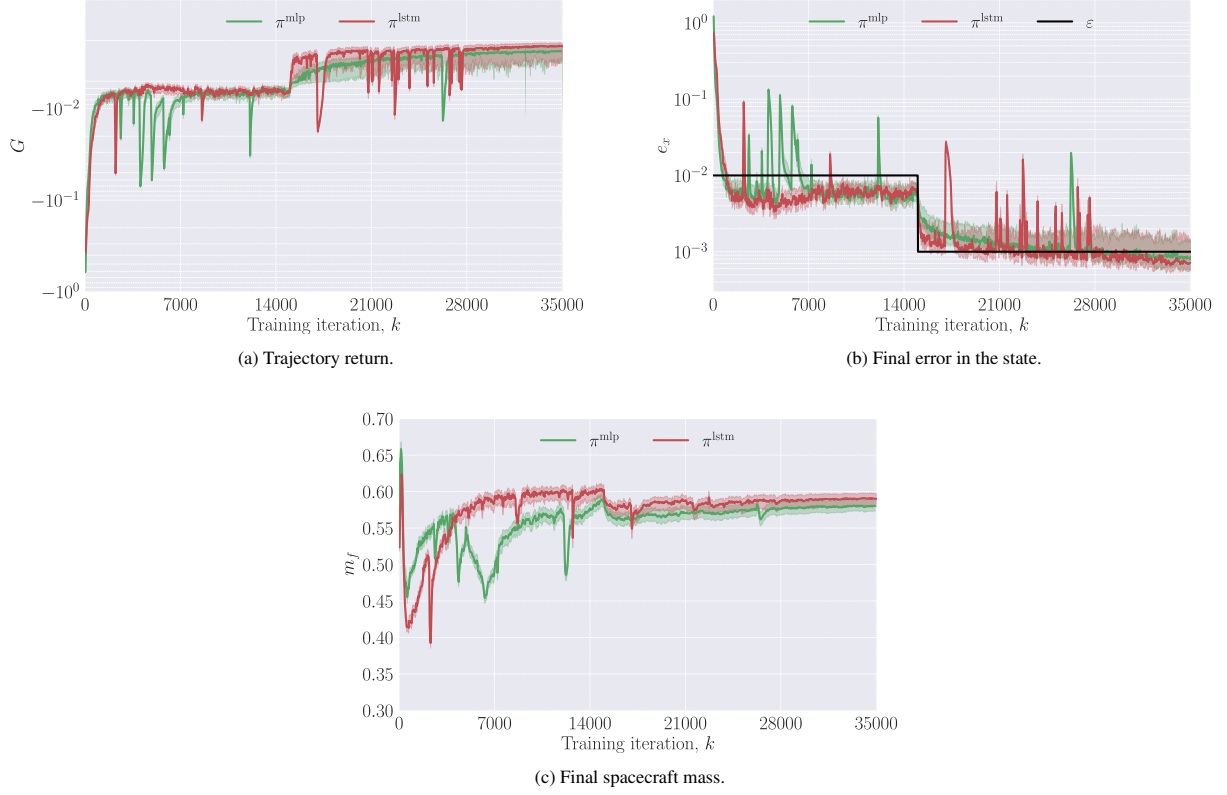


Figure 4: Performance trend of the two agents during training.

Table 4: Robust trajectories overview.

Agent	m_f , kg	e_r , 10^{-3}	e_v , 10^{-3}	G , 10^{-3}
π^{mlp}	586.91	0.29	0.00	-4.13
π^{lstm}	601.87	0.25	0.00	-3.98
indirect	603.91	0.00	0.00	-3.96

guidance law that can be run in real-time on the on-board computer. The closed-loop performance of the agents has been evaluated by a Monte Carlo campaign involving the deployment of the control laws in 500 randomly generated instances of the mission scenario with both dynamic and control uncertainties.

The corresponding spacecraft trajectories are shown in Fig. 6, with the dark-blue line indicating the robust reference trajectory, the light-blue arrows the nominal Δv s, and the gray lines the 500 Monte Carlo samples. The differences between each Monte Carlo trajectory and the corresponding reference trajectory are up-scaled by a factor of 4 for illustration purposes. Specifically, Fig. 6a is referred to agent π^{mlp} , while Fig. 6b to agent

π^{lstm} . The results of the Monte Carlo campaigns are summarized in Table 5, which reports, for each agent, the minimum, mean, and maximum value of the final spacecraft mass m_f and terminal state error e_x , the success rate SR, defined as the fraction of trajectories meeting the terminal rendezvous condition within the desired tolerance (10^{-3}), and the tolerance ε_B required to achieve a success rate equal to 68.3% (1σ), 95.5% (2σ), and 99.7% (3σ), respectively. The introduction of these three additional metrics was aimed at offering readers a more comprehensive means of assessing the effectiveness of the derived control policy. While the success rate provides an important measure of policy performance, it does not provide information about the

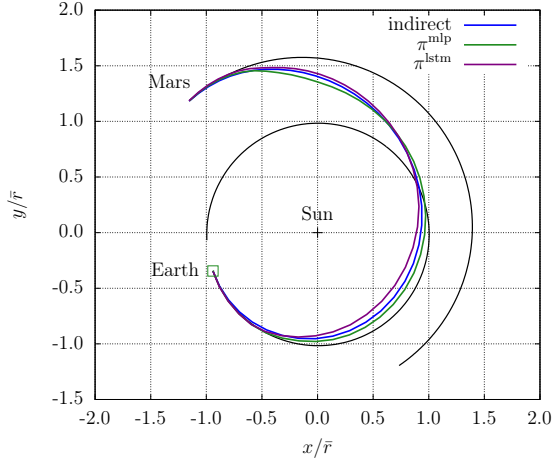


Figure 5: Robust trajectories obtained with the two agents.

extent to which other trajectories deviate from the desired accuracy. Consequently, these additional metrics contribute to a more in-depth evaluation of the control law.

The numerical results show that both closed-loop policies are able to compensate for both the considered uncertainties quite effectively. Indeed, the obtained success rates are rather high, over 73% with agent π^{mlp} and up to 85.4% with agent π^{lstm} . With either policy, the 95.5% of the trajectories meet the final rendezvous condition with an accuracy lower than 1.8 times the desired one. This value decreases to 1.6 with meta-RL. The final spacecraft mass is, on average, 4% lower than the optimal deterministic one with agent π^{mlp} and just 2% lower with agent π^{lstm} , thus confirming also the fuel-optimality of the closed-loop controls, which indeed follow an almost bang-off-bang law (Fig. 7). It is worth underlining that the results in Table 5 obtained with standard RL are worse than the ones reported in the previous study by the authors [32] since now the policy network is trained and deployed in an environment featuring state uncertainties and control errors simultaneously, while in the previous paper the network was trained (and evaluated) in environments with only one source of uncertainty at a time.

The results clearly state that meta-RL outperforms RL when considering the closed-loop performance as well, both in terms of average terminal constraint violation and final spacecraft mass. In this respect, Fig. 6 shows quite different behaviors between the two policies. Indeed, the Monte Carlo trajectories obtained with agent π^{lstm} feature a much greater dispersion in the initial phases of the transfer, which progressively

tends to shrink and eventually disappear as the spacecraft approaches Mars, as a consequence of the terminal rendezvous constraint. Conversely, the trajectories by agent π^{mlp} stay much closer to the reference robust trajectory during the whole mission. Nonetheless, π^{lstm} meets the terminal rendezvous condition with greater accuracy, as also highlighted by Fig. 8, which shows the distribution of the error $\mathbf{e}_r = (\mathbf{r}_f - \mathbf{r}_\delta)/|\mathbf{r}_\delta| = [e_x, e_y, e_z]^T$ on the spacecraft position at Mars encounter with the two agent networks. The three dashed lines identify the radii $\varepsilon_{\mathcal{B}}$ of the balls containing the 68.3%, 95.5%, and 99.7% of the solutions, respectively (i.e., the sigma-contours). Thus, results confirm that the recurrent-based network shows a greater adaptivity to the considered environment instance. The initial trajectory dispersion is a direct consequence of the closed-loop thrust laws, which, thanks to the history-dependent behavior, follow the optimal bang-off-bang pattern for the first 2/3 of the transfer and start compensating for the considered uncertainties just in the final phase of the mission.

As a final remark, it is worth underlining that this study specifically concentrated on examining the impact of different uncertainty distributions on a single nominal trajectory rather than exploring alternative trajectory geometries, flight durations, or boundary conditions, as its primary objective was to evaluate the performance of meta-RL in the context of robust interplanetary trajectory design. However, the capability of meta-RL to address these other aspects has been extensively investigated in a prior work by the authors (Ref.[29]).

6. Conclusion

This paper presented a meta-reinforcement learning (meta-RL) framework to deal with the robust design of low-thrust interplanetary trajectories in the presence of dynamic and control uncertainties. Two deep neural networks have been compared in the paper as control policies: a standard multi-layer perceptron (MLP), realizing a standard RL procedure, and a recurrent network with a long short-term memory (LSTM) layer, implementing a meta-RL procedure. Both networks have been trained via proximal policy optimization, a state-of-the-art policy gradient RL method, by using the same algorithm settings.

The numerical results show that meta-RL outperforms standard RL in the considered stochastic environment, achieving both better open-loop and closed-loop performance. Specifically, the use of meta-RL allows for a 2% gain on the average final spacecraft mass and more than 20% gain on the terminal constraint satisfaction accuracy.

Table 5: Results of the Monte Carlo simulations.

Agent	m_f , kg			e_x , 10^{-3}			ε_B , 10^{-3}			SR, %
	min	mean	max	min	mean	max	1σ	2σ	3σ	
π^{mlp}	539.60	579.53	615.91	0.08	0.80	3.25	0.90	1.76	3.15	73.8
π^{lstm}	515.00	590.97	632.68	0.09	0.67	5.26	0.70	1.62	4.30	85.4

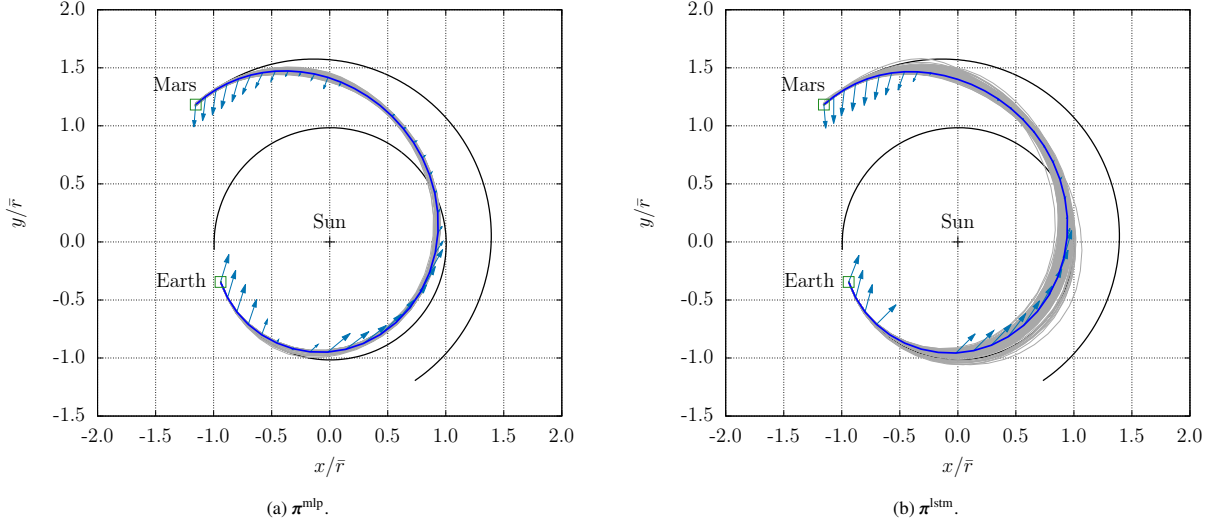


Figure 6: Monte Carlo trajectories obtained with the two agents.

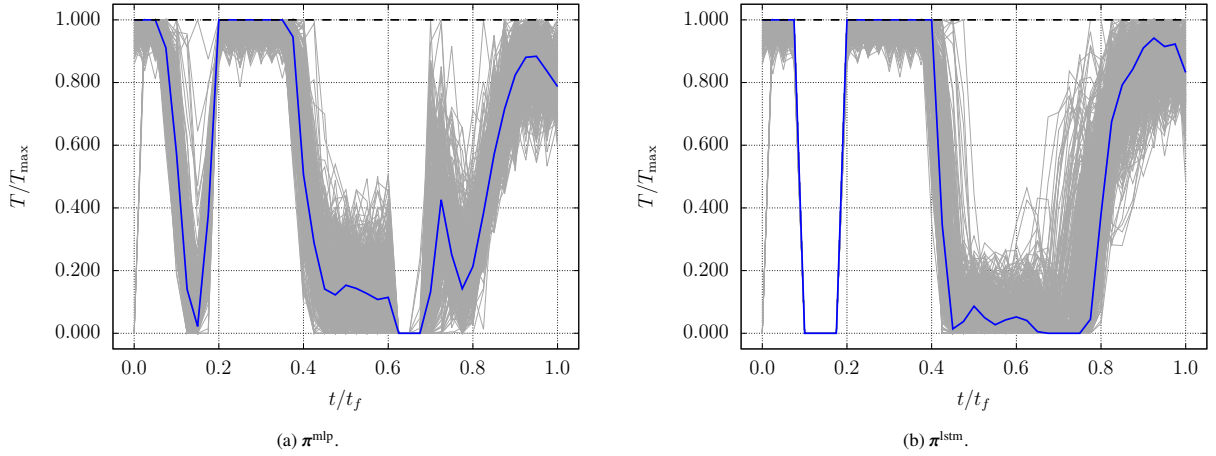


Figure 7: Closed-loop thrust laws obtained with the two agents.

Finally, it is worthwhile remarking that the meta-RL-based optimization methodology proposed in this paper is quite general and can be implemented, with the appropriate changes, to cope with a variety of spacecraft missions and state/control constraints. As an example, time-optimal and/or multi-revolution orbital trans-

fers could be investigated, too. Extensions to arbitrary stochastic dynamic models (e.g., with possibly complex non-Gaussian perturbations) and non-Markov scenarios (as in the presence of missed thrust events) are also straightforward. This is a major advantage with respect to other stochastic optimization techniques presented in

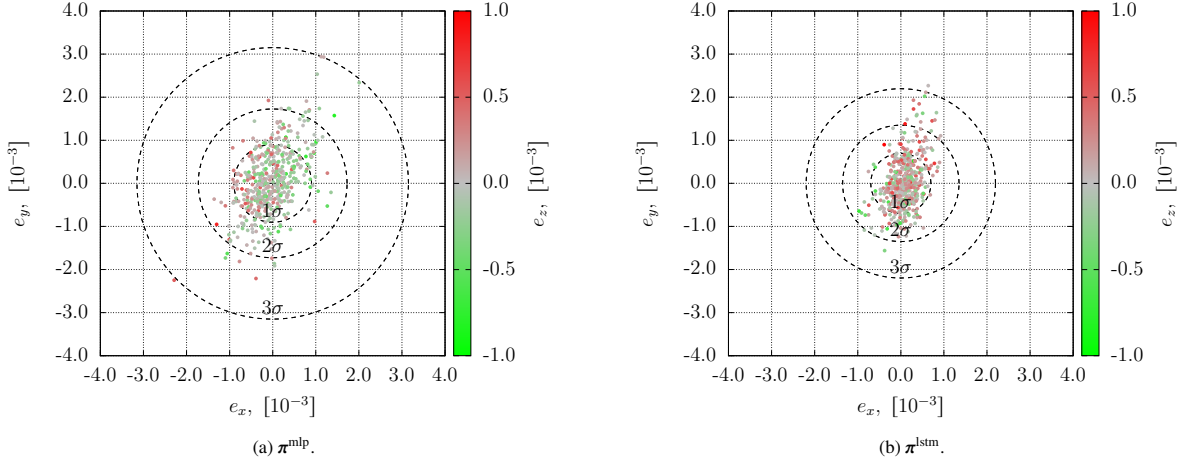


Figure 8: Distribution of the position errors at Mars encounter for the two agents.

the literature, which are based on ad-hoc extensions of traditional optimal control methods.

7. Appendix

This appendix is used to evaluate how the impulsive-maneuver approximation introduced in Sec. 2.2.2 to formulate the problem in Eq. (7) as an MDP impacts the accuracy of the obtained solutions. In this approximate model, an impulsive velocity variation equal to $\Delta \mathbf{v} = \frac{\mathbf{T}}{m} \Delta t$ applied at the beginning of each trajectory segment is used to approximate a low-thrust arc of time-length Δt with constant thrust direction and modulus $\mathbf{T}$. Let us define with $\mathbf{r}_{\text{low}}$, $\mathbf{v}_{\text{low}}$ and $\mathbf{r}_{\text{imp}}$, $\mathbf{v}_{\text{imp}}$ the position and velocity obtained at the end of the arc with the low-thrust and impulsive model, respectively. By supposing a constant spacecraft mass during the arc (the maximum mass variation is indeed around 2%), their difference in modulus is equal to

$$\|\mathbf{v}_{\text{imp}} - \mathbf{v}_{\text{low}}\| = \left\| \frac{\mathbf{T}}{m} \Delta t - \Delta \mathbf{v} \right\| = 0 \quad (39)$$

$$\|\mathbf{r}_{\text{imp}} - \mathbf{r}_{\text{low}}\| = \left\| \frac{1}{2} \frac{\mathbf{T}}{m} \Delta t^2 - \Delta \mathbf{v} \Delta t \right\| = \frac{1}{2} \frac{\|\mathbf{T}\|}{m} \Delta t^2 \quad (40)$$

Thus, the two models produce the same final velocity but different positions. The position error, given the data in Table 1, is approximately equal to 140×10^3 km, which is an error in line with the accuracy used in this study, that is, $\varepsilon = 10^{-3} \bar{r}$, with $\bar{r} = 149.6 \times 10^6$ km.

Funding Sources

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-

profit sectors.

References

- [1] A. E. Bryson, *Applied optimal control: Optimization, Estimation and Control*, Hemisphere Publishing Co., Washington, D.C., 1975.
- [2] C. Hargraves, S. Paris, Direct trajectory optimization using nonlinear programming and collocation, *Journal of Guidance, Control, and Dynamics* 10 (4) (1987) 338–342. doi:10.2514/3.20223.
- [3] H. Bock, K. Plitt, A multiple shooting algorithm for direct solution of optimal control problems, *IFAC Proceedings Volumes* 17 (2) (1984) 1603–1608. doi:10.1016/S1474-6670(17)61205-9.
- [4] M. D. Rayman, T. C. Fraschetti, C. A. Raymond, C. T. Russell, Coupling of system resource margins through the use of electric propulsion: Implications in preparing for the dawn mission to ceres and vesta, *Acta Astronautica* 60 (10-11) (2007) 930–938. doi:10.1016/j.actaastro.2006.11.012.
- [5] F. E. Laipert, J. M. Longuski, Automated missed-thrust propellant margin analysis for low-thrust trajectories, *Journal of Spacecraft and Rockets* 52 (4) (2015) 1135–1143. doi:10.2514/1.A33264.
- [6] R. Bellman, Dynamic programming, *Science* 153 (3731) (1966) 34–37. doi:10.1126/science.153.3731.34.
- [7] N. Ozaki, S. Campagnola, R. Funase, Tube stochastic optimal control for nonlinear constrained trajectory optimization problems, *Journal of Guidance, Control, and Dynamics* 43 (4) (2020) 645–655. doi:10.2514/1.6004363.
- [8] C. Greco, M. Di Carlo, M. Vasile, R. Epenoy, An intrusive polynomial algebra multiple shooting approach to the solution of optimal control problems, in: *69th International Astronautical Congress (IAC)*, Bremen, Germany, 2018.
- [9] C. Greco, M. Di Carlo, M. Vasile, R. Epenoy, Direct multiple shooting transcription with polynomial algebra for optimal control problems under uncertainty, *Acta Astronautica* 170 (2020) 224–234. doi:10.1016/j.actaastro.2019.12.010.
- [10] C. Greco, S. Campagnola, M. L. Vasile, Robust space trajectory design using belief stochastic optimal control, in: *AIAA Scitech 2020 Forum*, 2020. doi:10.2514/6.2020-1471.

- [11] C. Greco, M. Vasile, Closing the loop between mission design and navigation analysis, in: 71st International Astronautical Congress (IAC) - The CyberSpace Edition, 2020.
- [12] B. Benedikter, A. Zavoli, G. Colasurdo, S. Pizzurro, E. Cavallini, Autonomous upper stage guidance with robust splash-down constraint, in: 72nd International Astronautical Congress (IAC), Dubai, UAE, 2021.
- [13] B. Benedikter, A. Zavoli, Z. Wang, S. Pizzurro, E. Cavallini, Convex approach to covariance control with application to stochastic low-thrust trajectory optimization, *Journal of Guidance, Control, and Dynamics* (2022). doi:10.2514/1.G006806.
- [14] J. Ridderhof, P. Tsiotras, Minimum-fuel closed-loop powered descent guidance with stochastically derived throttle margins, *Journal of Guidance, Control, and Dynamics* 44 (3) (2021) 537–547. doi:10.2514/1.G005400.
- [15] B. Benedikter, A. Zavoli, Z. Wang, S. Pizzurro, E. Cavallini, Convex approach to stochastic control for autonomous rocket pinpoint landing, in: AAS/AIAA Astrodynamics Specialist Conference, Charlotte, NC, 2022.
- [16] H. A. Pierson, M. S. Gashler, Deep learning in robotics: a review of recent research, *Advanced Robotics* 31 (16) (2017) 821–835. doi:10.1080/01691864.2017.1365009.
- [17] B. Gaudet, R. Linares, R. Furfaro, Deep reinforcement learning for six degree-of-freedom planetary landing, *Advances in Space Research* 65 (7) (2020) 1723–1741. doi:10.1016/j.asr.2019.12.030.
- [18] L. Federici, B. Benedikter, A. Zavoli, Deep learning techniques for autonomous spacecraft guidance during proximity operations, *Journal of Spacecraft and Rockets* 58 (6) (2021) 1774–1785. doi:10.2514/1.A35076.
- [19] N. B. LaFarge, D. Miller, K. C. Howell, R. Linares, Autonomous closed-loop guidance using reinforcement learning in a low-thrust, multi-body dynamical environment, *Acta Astronautica* 186 (2021) 1–23. doi:10.1016/j.actaastro.2021.05.014.
- [20] D. Miller, J. A. Englander, R. Linares, Interplanetary low-thrust design using proximal policy optimization, *Advances in the Astronautical Sciences* 171 (2020) 1575–1592.
- [21] A. Rubinsztein, K. Bryan, R. Sood, F. Laipert, Using reinforcement learning to design missed thrust resilient trajectories, in: AAS/AIAA Astrodynamics Specialist Conference, no. AAS 20-453, Virtual Lake Tahoe, 2020.
- [22] S. Boone, S. Bonasera, J. W. McMahon, N. Bosanac, N. R. Ahmed, Incorporating observation uncertainty into reinforcement learning-based spacecraft guidance schemes, in: AIAA SCITECH 2022 Forum, 2022. doi:10.2514/6.2022-1765.
- [23] S. Hochreiter, A. S. Younger, P. R. Conwell, Learning to learn using gradient descent, in: *Artificial Neural Networks - ICANN 2001*, Springer Berlin Heidelberg, Berlin, Germany, 2001, pp. 87–94.
- [24] B. Gaudet, R. Linares, R. Furfaro, Six degree-of-freedom body-fixed hovering over unmapped asteroids via lidar altimetry and reinforcement meta-learning, *Acta Astronautica* 172 (2020) 90–99. doi:10.1016/j.actaastro.2020.03.026.
- [25] B. Gaudet, R. Linares, R. Furfaro, Terminal adaptive guidance via reinforcement meta-learning: Applications to autonomous asteroid close-proximity operations, *Acta Astronautica* 171 (2020) 1–13. doi:10.1016/j.actaastro.2020.02.036.
- [26] A. Scorsoglio, A. D'Ambrosio, L. Ghilardi, B. Gaudet, F. Curti, R. Furfaro, Image-based deep reinforcement meta-learning for autonomous lunar landing, *Journal of Spacecraft and Rockets* 59 (1) (2021) 153–165. doi:10.2514/1.A35072.
- [27] L. Federici, A. Scorsoglio, L. Ghilardi, A. D'Ambrosio, B. Benedikter, A. Zavoli, R. Furfaro, Image-based meta-reinforcement learning for autonomous guidance of an asteroid impactor, *Journal of Guidance, Control, and Dynamics* 45 (11) (2022) 2013–2028. doi:10.2514/1.G006832.
- [28] L. Federici, A. Scorsoglio, A. Zavoli, R. Furfaro, Optical 6-dof guidance and navigation of an asteroid impactor via meta-reinforcement learning, in: 2022 AAS/AIAA Astrodynamics Specialist Conference, Charlotte (NC), USA, 2022.
- [29] L. Federici, A. Scorsoglio, A. Zavoli, R. Furfaro, Meta-reinforcement learning for adaptive spacecraft guidance during finite-thrust rendezvous missions, *Acta Astronautica* 201 (2022) 129–141. doi:10.1016/j.actaastro.2022.08.047.
- [30] G. Lantoine, R. P. Russell, A hybrid differential dynamic programming algorithm for constrained optimal control problems. part 2: Application, *Journal of Optimization Theory and Applications* 154 (2) (2012) 418–442. doi:10.1007/s10957-012-0038-1.
- [31] E. Schiassi, A. D'Ambrosio, K. Drozd, F. Curti, R. Furfaro, Physics-informed neural networks for optimal planar orbit transfers, *Journal of Spacecraft and Rockets* 59 (3) (2022) 834–849. doi:10.2514/1.A35138.
- [32] A. Zavoli, L. Federici, Reinforcement learning for robust trajectory design of interplanetary missions, *Journal of Guidance, Control, and Dynamics* 44 (8) (2021) 1440–1453. doi:10.2514/1.G005794.
- [33] J. A. Sims, S. N. Flanagan, Preliminary design of low-thrust interplanetary missions, *Advances in the Astronautical Sciences* 103 (1) (2000) 583–592.
- [34] R. R. Bate, D. D. Mueller, J. E. White, *Fundamentals of Astrodynamics*, Dover, NY, 1971.
- [35] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, *arXiv preprint arXiv:1707.06347* (2017).
- [36] J. Schulman, P. Moritz, S. Levine, M. Jordan, P. Abbeel, High-dimensional continuous control using generalized advantage estimation, *arXiv preprint arXiv:1506.02438* (2015).
- [37] N. Schweighofer, K. Doya, Meta-learning in reinforcement learning, *Neural Networks* 16 (1) (2003) 5–9.
- [38] J. X. Wang, Z. Kurth-Nelson, D. Tirumala, H. Soyer, J. Z. Leibo, R. Munos, C. Blundell, D. Kumaran, M. Botvinick, Learning to reinforcement learn, *arXiv preprint arXiv:1611.05763* (2016).
- [39] I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press, 2016, <http://www.deeplearningbook.org>.
- [40] S. Hochreiter, J. Schmidhuber, Long Short-Term Memory, *Neural Computation* 9 (8) (1997) 1735–1780. doi:10.1162/neco.1997.9.8.1735.
- [41] P. Moritz, R. Nishihara, S. Wang, A. Tumanov, R. Liaw, E. Liang, M. Elibol, Z. Yang, W. Paul, M. I. Jordan, et al., Ray: A distributed framework for emerging ai applications, in: 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18), 2018, pp. 561–577.
- [42] L. Federici, B. Benedikter, A. Zavoli, Eos: a parallel, self-adaptive, multi-population evolutionary algorithm for constrained global optimization, in: 2020 IEEE Congress on Evolutionary Computation (CEC), 2020, pp. 1–10. doi:10.1109/CEC48606.2020.9185800.
- [43] G. Colasurdo, A. Zavoli, A. Longo, L. Casalino, F. Simeoni, Tour of jupiter galilean moons: Winning solution of gtoc6, *Acta Astronautica* 102 (2014) 190–199. doi:10.1016/j.actaastro.2014.06.003.